Laboratory Report – Logger and Parentheses Validator

Student: Adrian Frunza (FAF 253) **Course:** Object-Oriented Programming (Lab 1)
Repository: `https://github.com/adrianfza/OOP`
Programming Language: C++ (C++17)
Instructor: Dominic Flocea **Date:** October 2026

1. Program Structure

What is the responsibility of `Logger`?

The `Logger` class is responsible for formatting diagnostic events, filtering messages based on a configured severity level, and printing the output to the console. It manages shared application-wide configurations (level and format) while allowing individual components to identify themselves via an instance name.

What is the responsibility of `ParenthesesValidator`?

The `ParenthesesValidator` class evaluates whether a given string of parentheses is balanced and ordered correctly. It processes the input linearly, uses a stack-based algorithm to track opening brackets, and reports its progress and final boolean verdict using the logger.

How do the two classes collaborate?

The classes collaborate via object composition. `ParenthesesValidator` encapsulates a private instance of `Logger`. During the validation process, the validator calls the logger's methods (`trace`, `info`, `warn`, `error`) to report state changes. The validator does not inherit from the logger; it delegates the logging responsibility to it.

2. Classes and Objects

What is a class in your implementation?

In this implementation, a class is a blueprint that encapsulates state (data members) and behavior (member functions). For example, the `Logger` class bundles state like `name`, `level`, and `format` with behaviors like `log()` and `render()`.

Give an example of an object created from a class.

Inside the `ParenthesesValidator` constructor, a concrete `Logger` object is instantiated:

```
ParenthesesValidator() : logger("Parentheses") {}
```

Here, `logger` is an actual object existing in memory with its own identity (its `name` attribute is set to `"Parentheses"`).

What state does each object store?

- `Logger` **object:** Stores its instance attribute `name` (a string representing the module).
- `ParenthesesValidator` **object:** Stores its private `logger` instance. During the execution of `isValid()`, it also utilizes local state (a `std::vector<std::size_t>` stack) to track the indices of unmatched opening parentheses.

3. Static and Instance Members

Which members of `Logger` are static, and why?

The members `level` and `format` are static, along with their respective setter methods (`setLevel` and `setFormat`). They are static because they represent global, application-wide configurations. If we want to change the logging threshold from `INFO` to `TRACE`, we want this rule to apply universally without needing to update every individual logger instance one by one.

Which member belongs to an individual logger instance?

The `name` string belongs to an individual instance. This allows different subsystems to tag their logs independently (e.g., `"Parentheses"`, `"Database"`) while sharing the static formatting and level rules.

What happens when the logging level is changed? Does it affect every logger instance? Explain why.

Changing the logging level instantly affects every `Logger` instance. Static variables reside in the program's static data segment rather than within the memory layout of individual objects. Because all objects reference the exact same memory location for `level`, a single update alters the behavior globally.

```
// Affects all loggers program-wide
Logger::setLevel(LogLevel::ERROR);

// Inside Logger::log(), this check references the shared static variable:
if (static_cast<int>(msgLevel) < static_cast<int>(level)) return;
```

4. Methods and Responsibilities

What is the responsibility of the `log` method?

The `log` method has three main responsibilities: 1. **Filtering:** It aborts execution if the requested message level is below the statically configured threshold. 2. **Formatting:** It passes the message to the `render()` helper function to replace placeholders (e.g., `%t`, `%n`) with runtime values. 3. **Output:** It prints the finalized string to standard output.

Why does `ParenthesesValidator` use a logger instead of implementing logging itself?

This adheres to the Single Responsibility Principle. `ParenthesesValidator` is strictly an algorithmic parser. It should not be burdened with string templating, fetching system time, or knowing how to write to the console. By using a logger, we decouple the validation logic from the output logic.

Identify one example of a method call between objects or classes in your program.

Inside `ParenthesesValidator::isValid`, the validator invokes a method on its logger object:

```
logger.trace("Opening parenthesis found at index " + std::to_string(i));
```

5. Testing and Correctness

Give one valid and one invalid input for the parentheses validator.

- **Valid input:** `"(()((()()))())"` returns `true`.
- **Invalid input:** `")((()))"` returns `false`.

Explain how the program determines whether the input is valid.

The program scans the string sequentially using a stack (`std::vector`). 1. When it encounters `'( '`, it pushes the index onto the stack. 2. When it encounters `') '`, it checks if the stack is empty. If empty, the parenthesis is unmatched (invalid). If not empty, it pops the top index from the stack, representing a successful match. 3. After the loop, it checks if the stack is entirely empty. If any indices remain, it means opening brackets were never closed (invalid). 4. If the loop completes with an empty stack and no early exits, the string is valid.

Describe one test that helped you find or prevent an error.

Testing the input `"("`. If the algorithm only checked for invalid closing brackets, it would mistakenly return `true` for this input because it never encounters a `') '`. This test verifies that the final post-loop check (`!stack.empty()`) is working correctly to catch unclosed opening brackets.

6. Reflection

What was the most important OOP concept you applied in this laboratory?

The distinction between static and instance scopes alongside object composition. Using static variables for `format` and `level` perfectly modeled a shared application state, while instance variables gave individual components identity. Composition allowed `ParenthesesValidator` to utilize complex logging behavior without polluting its own logic.

If you had more time, what would you improve in your design, and why?

I would implement an abstract interface for the output sink (e.g., an `IOutputWriter` interface) so that the `Logger` could easily be configured to write to a file, a network socket, or memory instead of being hardcoded to `std::cout`. I would also expand the validator to handle multiple bracket types (`{}`, `[]`) by tracking bracket pairs in a dictionary map.